

EPAM Systems – MERN FullStack Interview Cheatsheet

Everything in one place: definitions to say in interviews, ELI5 explanations, bite-size bullets, code templates, and common twisters. Crafted from our prep thread, organized for last-minute revision.

Scope: React (Hooks, keys, useMemo/useCallback), Node.js (event loop, Express), Microservices (Saga, Circuit Breaker, Transactional Outbox), Redis (RDB vs AOF, eviction, Streams), MongoDB (modeling, indexes, aggregation, change streams, TTL, sharding), AWS (ALB vs NLB vs API Gateway, IAM, SG vs NACL), plus DSA patterns and handy snippets.

Tip: Practice saying the green “Definition” lines out loud—they’re phrased to be concise in interviews.

React Essentials

Definition (Interview-safe): Hooks must be called at the top level of a function component or custom hook; never in loops/conditions. Keys must be stable/unique to preserve component identity across renders.

ELI5: React re-renders by comparing the old and new UI trees. Keys are like name tags so React knows which list row is which. Hooks are lined up by the order they’re called; changing the order confuses React.

useMemo vs useCallback

Definition (Interview-safe): useMemo caches a computed value; useCallback caches a function reference. Use them to avoid expensive recalculations or to keep props stable for memoized children.

When to use: • Heavy derived data (filtering/sorting large arrays) → useMemo. • Passing callbacks/objects into React.memo children → useCallback/useMemo to keep stable references.

Twister: Overusing memoization can slow down initial render and add complexity. Don’t use them to fix logic bugs—only for performance.

```
// useMemo: heavy compute
const total = useMemo(() => items.reduce((s,x)=>s+x.price,0), [items]);

// useCallback: stable function for memoized child
const onSave = useCallback((mode) => saveDoc(id, mode), [id]);
```

Keys & Lists

Definition (Interview-safe): Keys give list items a stable identity across renders. Use data IDs, not array indexes, when order can change.

Twister: Using array index as key in a re-orderable list can cause state to ‘jump’ rows (e.g., inputs losing cursor/values).

Effects & Dependencies

Definition (Interview-safe): If an effect reads a reactive value (props/state/functions), include it in the dependency array so the effect re-runs with fresh values.

```
useEffect(() => {
  let cancel = false;
```

```

    fetch('/api?q=' + query).then(r=>r.json()).then(d=> !cancel && setData(d));
    return () => { cancel = true }
  }, [query]);

```

Node.js & Express

Definition (Interview-safe): Node.js runs JS on a single thread with a multi-phase event loop; offloads I/O to the system. Express runs middleware in the order you register it; error handler with 4 args must be last.

ELI5: Think of a kitchen: the chef (JS thread) handles orders quickly and sends slow tasks (I/O) to helpers. Don't give the chef a 10-minute task (CPU loop) or the line stalls.

Event Loop Phases (Node)

Timers → Pending Callbacks → Idle/Prepare → Poll → Check (setImmediate) → Close Callbacks; microtasks (nextTick/promise jobs) run between turns. After I/O, setImmediate often runs before setTimeout(0).

Twister: process.nextTick runs before promise microtasks and can starve I/O if you schedule too many.

```

setTimeout(()=>console.log('timeout'),0);
setImmediate(()=>console.log('immediate'));
require('fs').readFile(__filename, ()=>{
  setTimeout(()=>console.log('t-in-io'),0);
  setImmediate(()=>console.log('i-in-io')); // usually first after I/O
});

```

Express Middleware Order + Error Handling

```

app.use(helmet());
app.use(rateLimiter);
app.use('/api', auth, apiRouter);

// 404
app.use((req,res)=>res.status(404).json({error:'Not found'}));

// error handler (must be last; 4 args)
app.use((err,req,res,next)=>{
  console.error(err);
  res.status(500).json({error:'Internal'});
});

```

Microservices – Core Patterns

Circuit Breaker

Definition (Interview-safe): Wrap remote calls so repeated failures open the breaker and fail fast; after a cool-down, allow limited test calls (half-open). Prevents cascades.

ELI5: Like a home fuse: too many failures → trip; wait; test; then resume if healthy.

```

// pseudo-usage
await breaker.call(() => httpClient.get('/payments'));

```

Saga (Distributed Workflow)

Definition (Interview-safe): Split a cross-service business action into local transactions; on failure, run compensating actions. Implement via choreography (events) or orchestration (coordinator).

Transactional Outbox (No Dual-Writes)

Definition (Interview-safe): In the same DB transaction as your business write, insert an 'outbox' event row. A publisher/CDC reads and emits events, marking rows sent. Guarantees DB and events stay consistent.

```
// Mongo (Mongoose) sketch
await session.withTransaction(async ()=>{
  await Orders.create([{_id: oid, ...}], {session});
  await Outbox.create([{ type:'OrderPlaced', payload:{ oid }, status:'NEW'}], {session});
});
// publisher polls NEW -> publish -> mark SENT
```

Redis – What Matters for Interviews

Persistence

Definition (Interview-safe): RDB (Redis Database snapshot) writes point-in-time snapshots; AOF (Append-Only File) logs every write for better durability. Many setups enable both.

Eviction & TTL

Definition (Interview-safe): Set maxmemory and a policy (e.g., allkeys-lru, volatile-ttl). Under memory pressure, Redis evicts per policy. Use TTLs for session/cache keys.

```
SET session:123 "... " EX 3600
CONFIG SET maxmemory 1gb
CONFIG SET maxmemory-policy allkeys-lru
```

Streams vs Pub/Sub

Definition (Interview-safe): Pub/Sub is fire-and-forget (no history). Streams store messages with IDs + consumer groups (replay/at-least-once), ideal for work queues.

MongoDB – Modeling, Indexing, Aggregation

Embed vs Reference

Definition (Interview-safe): Embed small, bounded data read with the parent (fewer queries). Reference when child grows unbounded, is shared, or updated independently (scales better).

Indexes You'll Actually Use

- Single, Compound (left-prefix rule), Multikey (arrays), Text, Hashed, TTL, Partial, Wildcard.

Aggregation Pipeline (core stages)

\$match → \$project → \$group → \$sort → \$limit → \$lookup → \$facet → \$merge/\$out. Push \$match early; index join keys for \$lookup.

Change Streams

Definition (Interview-safe): Subscribe to DB/collection/cluster changes without tailing the oplog—great for cache invalidation and real-time feeds (replica set needed).

Transactions

Definition (Interview-safe): Use session transactions only when you need multi-doc atomicity; otherwise rely on single-doc atomic updates.

TTL Indexes

Definition (Interview-safe): Auto-expire docs by duration or at a fixed time; perfect for sessions/logs.

Sharding

Definition (Interview-safe): Pick a shard key with high cardinality, good distribution, and aligned with query patterns; avoid monotonic hot-spot keys.

AWS – What a MERN Full-Stack Dev Should Know

IAM Roles vs Users

Definition (Interview-safe): Roles are assumed identities with temporary credentials (for apps/services). Users have long-term credentials (for humans). Prefer roles for workloads.

Security Groups vs NACLs

Definition (Interview-safe): SGs are stateful, instance/ENI-level firewalls. NACLs are stateless, subnet-level allow/deny lists. Use SGs daily; NACLs as subnet guardrails.

ALB vs NLB vs API Gateway

- ALB (Layer 7): HTTP/gRPC, path/host routing, WebSockets; great for web/container APIs.
- NLB (Layer 4): TCP/UDP/TLS, extreme throughput, static IPs/Elastic IPs.
- API Gateway: managed API front door (auth, throttling, usage plans), ideal for serverless/public APIs.

Hosting a React SPA

Build → S3 (static hosting) → CloudFront (CDN). Backend on ECS/EC2 behind ALB; secure with IAM roles, SGs, and WAF as needed.

DSA Patterns (Fast Recall)

- Sliding Window (longest substring without repeat)

```
function lengthOfLongestSubstring(s){
  let left=0,best=0,seen=new Map();
  for(let right=0; right<s.length; right++){
    const ch=s[right];
    if(seen.has(ch) && seen.get(ch)>=left) left=seen.get(ch)+1;
    seen.set(ch,right);
    best=Math.max(best,right-left+1);
  }
  return best;
}
```

- Binary Search on Answer (e.g., Koko Eating Bananas)

```
function minEatingSpeed(piles,h){
  let lo=1, hi=Math.max(...piles);
```

```

const ok=k => piles.reduce((t,p)=>t+Math.ceil(p/k),0) <= h;
while(lo<hi){ const mid=(lo+hi>>1); ok(mid)?hi=mid:lo=mid+1; }
return lo;
}

```

- LRU Cache (Map order trick)

```

class LRU {
  constructor(cap){ this.cap=cap; this.m=new Map(); }
  get(k){ if(!this.m.has(k)) return -1; const v=this.m.get(k);
    this.m.delete(k); this.m.set(k,v); return v; }
  put(k,v){ if(this.m.has(k)) this.m.delete(k);
    this.m.set(k,v); if(this.m.size>this.cap){ const [f]=this.m.keys(); this.m.delete(f); } }
}

```

Common Twisters (Interview Gotchas)

- React: Using array index as key in a sortable list → state jumps. Fix: use stable IDs.
- useEffect: Missing a dep causes stale reads or missed updates. Include everything you read; memoize functions with useCallback.
- Node: Blocking CPU on the main thread stalls all requests; offload or worker threads.
- Redis: No TTL + wrong eviction policy → memory errors. Configure maxmemory + policy + expirations.
- Mongo: \$lookup without index → slow joins. Index join fields; limit unwinds early.
- AWS: Using users/keys in code instead of roles; mixing up SG (stateful) vs NACL (stateless).

Sources & Official Docs (quick verify)

- [React Rules of Hooks](#)
- [useMemo](#)
- [useCallback](#)
- [Lists & Keys / Reconciliation](#)
- [Node Event Loop / nextTick / setImmediate](#)
- [Express Error Handling](#)
- [Redis Persistence \(RDB/AOF\)](#)
- [Redis Eviction Policy](#)
- [AWS IAM Roles](#)
- [AWS SG vs NACL](#)
- [ALB vs NLB \(AWS\)](#)
- [API Gateway Throttling/Usage Plans](#)
- [S3 + CloudFront for SPA](#)
- [MongoDB Embedding vs References](#)
- [MongoDB Index Types](#)
- [MongoDB Aggregation Pipeline](#)
- [MongoDB Change Streams](#)

- [MongoDB Transactions](#)
- [MongoDB TTL Indexes](#)
- [MongoDB Choose a Shard Key](#)
- [Circuit Breaker](#)
- [Transactional Outbox](#)
- [Saga Pattern](#)
- [CQRS](#)

Prepared for fast revision. Good luck — you've got this!